GOVERNANÇA E COMPLIANCE AMBIENTAL

EcoCompliance API

Pipeline de CI/CD, containerização e deploy em staging e produção.

FIAP — Tecnologia em Análise e Desenvolvimento de Sistemas. Disciplina de DevOps, fase “Navegando pelo mundo DevOps”. A aplicação veio da disciplina de Java Advanced (microsserviços com Spring).

Integrantes

Nome	RM
Évelyn Santos Rodrigues	565919
Fagner Nunes Lopes	564220
Lucas Fernando da Silva	563626
Raphael Salomão Chamma	564765
Ricardo Negrão da Veiga	564106

1. Pipeline

A ferramenta é o GitHub Actions. O arquivo versionado é `.github/workflows/ci-cd.yml`, no repositório `github.com/fagnerlopes/ecocompliance-api`. Push e pull request nas branches **main** e **develop** disparam o fluxo.

Etapa	Quando	O que executa
Build	Todo push e pull request	mvn -B -DskipTests package com JDK 21
Testes	Em paralelo com o build	mvn -B test — 166 testes, 0 falhas
Somente develop	Pull request com destino main	Confere se a origem é a develop
Imagem	Push em develop ou main	az acr build publica a imagem no ACR
Deploy staging	Push em develop	Atualiza o App Service ecocompliance-staging
Deploy produção	Push em main	Atualiza o App Service ecocompliance-production

No pull request rodam Build e Testes. Quando o destino é a main, entra também o job Somente develop. A publicação da imagem e o deploy acontecem no push: a **develop** atualiza o staging e a **main** atualiza a produção. A imagem vai para o Azure Container Registry `acrecocompliance.azurecr.io`, com a tag do commit e a tag latest. O App Service correspondente passa a usar essa tag.

Staging: `https://ecocompliance-staging.azurewebsites.net`
Produção: `https://ecocompliance-production.azurewebsites.net`

.github/workflows/ci-cd.yml

```
name: CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

env:
  REGISTRY: acrecocompliance.azurecr.io
  IMAGE: ecocompliance-api
  RESOURCE_GROUP: rg-ecocompliance

jobs:
  build:
    name: Build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
    - uses: actions/setup-java@v4
      with:
        distribution: temurin
        java-version: "21"
        cache: maven
    - name: Compilar o pacote
      run: mvn -B -DskipTests package

test:
  name: Testes
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with:
        distribution: temurin
        java-version: "21"
        cache: maven
    - name: Executar a suíte
      run: mvn -B test

somente-develop:
  name: Somente develop
  if: github.event_name == 'pull_request' && github.base_ref == 'main'
  runs-on: ubuntu-latest
  steps:
    - name: Recusar pull request que não venha da develop
      run: |
        if [ "${{ github.head_ref }}" != "develop" ]; then
          echo "A main só aceita pull request vindo da branch develop. Esta origem é ${{ github.head_ref }}."
          exit 1
        fi
        echo "Origem aceita: develop."

imagem:
  name: Imagem
  needs: [build, test]
  if: github.event_name == 'push' && (github.ref == 'refs/heads/develop' || github.ref == 'refs/heads/main')
  runs-on: ubuntu-latest
  outputs:
    tag: ${{ github.sha }}
  steps:
    - uses: actions/checkout@v4
    - uses: azure/login@v2
      with:
        creds: ${{ secrets.AZURE_CREDENTIALS }}
    - name: Publicar a imagem no ACR
      run: |
        az acr build \
          --registry acrecocompliance \
          --image "${IMAGE}:${{ github.sha }}" \
          --image "${IMAGE}:latest" \
          .

deploy-staging:
  name: Deploy staging
  needs: imagem
  if: github.ref == 'refs/heads/develop'
  runs-on: ubuntu-latest
  steps:
    - uses: azure/login@v2
      with:
        creds: ${{ secrets.AZURE_CREDENTIALS }}
    - name: Atualizar o App Service de staging
      run: |
        az webapp config container set \
          --resource-group "$RESOURCE_GROUP" \
          --name ecocompliance-staging \
          --container-image-name "${REGISTRY}/${IMAGE}:${{ needs.imagem.outputs.tag }}" \
          --container-registry-url "https://${REGISTRY}" \
          -o none
        az webapp config set \
          --resource-group "$RESOURCE_GROUP" \
          --name ecocompliance-staging \
          --generic-configurations '{"acrUseManagedIdentityCreds": true}' \
          -o none
        az webapp restart --resource-group "$RESOURCE_GROUP" --name ecocompliance-staging

deploy-production:
  name: Deploy produção
  needs: imagem
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: azure/login@v2
      with:
        creds: ${{ secrets.AZURE_CREDENTIALS }}
```

```
      - name: Atualizar o App Service de produção
        run: |
          az webapp config container set \
            --resource-group "$RESOURCE_GROUP" \
            --name ecocompliance-production \
            --container-image-name "${REGISTRY}/${IMAGE}:${{ needs.imagem.outputs.tag }}" \
            --container-registry-url "https://${REGISTRY}" \
            -o none
          az webapp config set \
            --resource-group "$RESOURCE_GROUP" \
            --name ecocompliance-production \
            --generic-configurations '{"acrUseManagedIdentityCreds": true}' \
            -o none
          az webapp restart --resource-group "$RESOURCE_GROUP" --name ecocompliance-production
```

2. Docker

A arquitetura local sobe a API e o Oracle XE 21 na rede `ecocompliance-net`. A API alcança o banco pelo host `oracle-db`. Os dados ficam no volume `oracle-data`. No Azure, o mesmo Dockerfile vira a imagem publicada no ACR e executada pelos dois App Services.



No Compose a API fala com o Oracle pela rede. No Azure a imagem sai do Actions, passa pelo ACR e chega ao App Service.



Figura. Arquitetura local (Compose) e caminho da imagem até o App Service.

O `Dockerfile` é multi-stage. A etapa `maven:3.9-eclipse-temurin-21` gera o jar com `mvn clean package -DskipTests`. A suíte permanece no job Testes. A imagem final, `eclipse-temurin:21-jre-alpine`, cria o usuário `spring`, copia o jar e escuta a porta 8080. O Compose etiqueta essa imagem como `ecocompliance-api:${APP_ENV}`.

O pipeline autentica na Azure com o secret `AZURE_CREDENTIALS`. Ele não lê arquivo `.env`. Staging e produção publicados são os App Services.

Ambiente	Endereço
Staging	https://ecocompliance-staging.azurewebsites.net
Produção	https://ecocompliance-production.azurewebsites.net

Comandos usados para construir a imagem e conferir os containers:

```
docker build -t ecocompliance-api:local .
docker images ecocompliance-api
docker ps -a --filter name=ecocompliance
```

Saída registrada nesta máquina em 13/10/2026:

```
docker images ecocompliance-api
REPOSITORY          TAG          IMAGE ID       SIZE
ecocompliance-api   production   d08ee1b943f8   269MB
ecocompliance-api   staging      c87d0808dbe8   269MB

docker ps -a --filter name=ecocompliance
```

```
NAMES                          STATUS    PORTS
ecocompliance-api-staging      Exited    0.0.0.0:8081->8080/tcp
ecocompliance-oracle-staging   Exited    0.0.0.0:1522->1521/tcp
ecocompliance-api              Exited    0.0.0.0:8080->8080/tcp
ecocompliance-oracle           Exited    0.0.0.0:1521->1521/tcp
```

Dockerfile

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app

COPY pom.xml .
RUN mvn dependency:go-offline -B

COPY src ./src
RUN mvn clean package -DskipTests -B

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring

COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080

ENV JAVA_OPTS=""
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

docker-compose.yml

```
services:
  oracle-db:
    image: gvenzl/oracle-xe:21-slim
    container_name: ${ORACLE_CONTAINER_NAME:-ecocompliance-oracle}
    ports:
      - "${ORACLE_PORT:-1521}:1521"
    environment:
      ORACLE_PASSWORD: ${ORACLE_PASSWORD:-oracle}
      APP_USER: ${DB_USERNAME:-ecocompliance}
      APP_USER_PASSWORD: ${DB_PASSWORD:-ecocompliance123}
    volumes:
      - oracle-data:/opt/oracle/oradata
    healthcheck:
      test: ["CMD", "healthcheck.sh"]
      interval: 30s
      timeout: 10s
      retries: 10
      start_period: 90s
    networks:
      - ecocompliance-net

  ecocompliance-api:
    build: .
    image: ecocompliance-api:${APP_ENV:-local}
    container_name: ${API_CONTAINER_NAME:-ecocompliance-api}
    ports:
      - "${API_PORT:-8080}:8080"
    depends_on:
      oracle-db:
        condition: service_healthy
    environment:
      APP_ENV: ${APP_ENV:-local}
      DB_URL: jdbc:oracle:thin:@oracle-db:1521/XEPDB1
      DB_USERNAME: ${DB_USERNAME:-ecocompliance}
      DB_PASSWORD: ${DB_PASSWORD:-ecocompliance123}
      JWT_SECRET: ${JWT_SECRET:-Y2hhdmUtc2VjcmV0YS1lY29jb21wbGlhbmNlLWFwaS1maWFwLTIwMjUtbXVpdG8tbG9uZ2EtZS1zZWd1cmE=}
      JAVA_OPTS: "-Xms256m -Xmx512m"
    networks:
      - ecocompliance-net

volumes:
  oracle-data:

networks:
  ecocompliance-net:
    driver: bridge
```

3. Evidências

Capturas de 10/10/2026. O push da develop publicou o staging e o push da main publicou a produção. O job Testes dessa execução registrou 166 testes e nenhuma falha. Os passos visíveis são `Compilar o pacote` e `Executar a suíte`. Na mesma tela, Imagem e o deploy do ambiente estão concluídos.

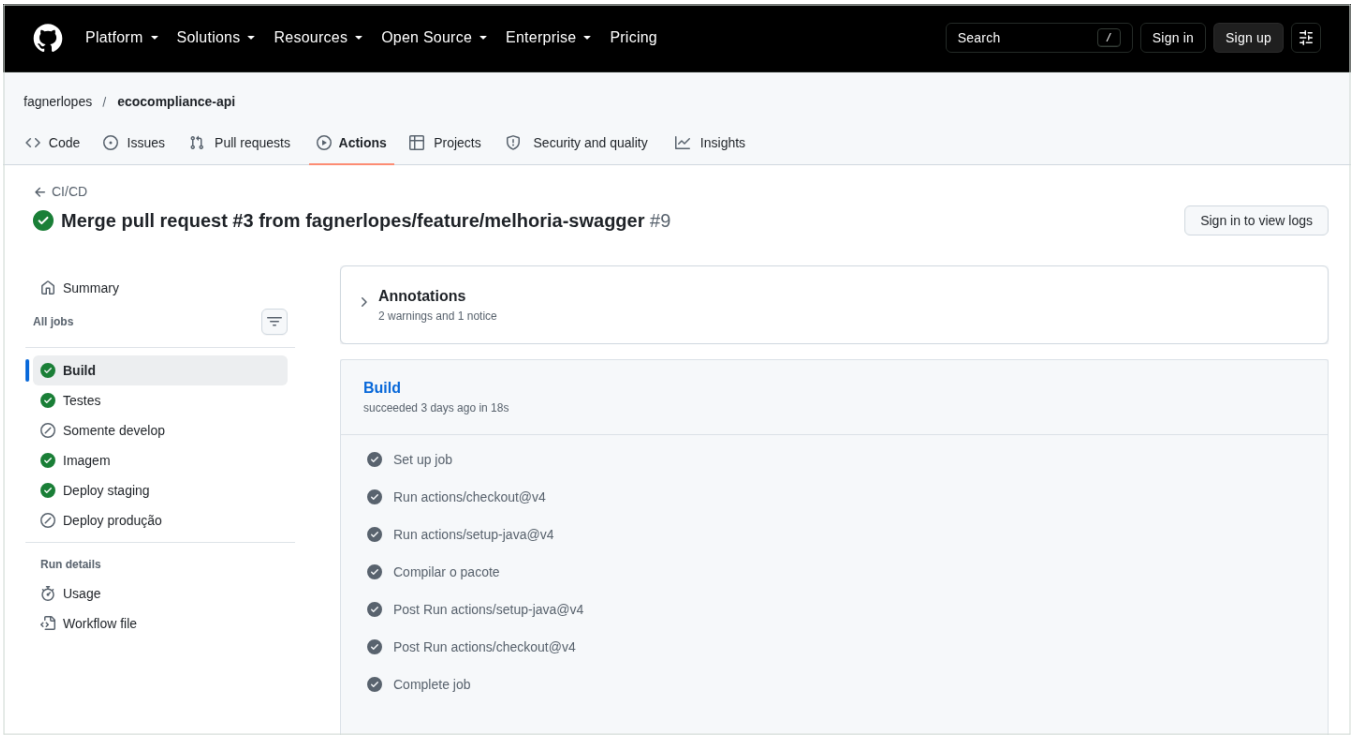


Figura 1. Push na develop: Build aberto em Compilar o pacote. Testes, Imagem e Deploy staging concluídos.

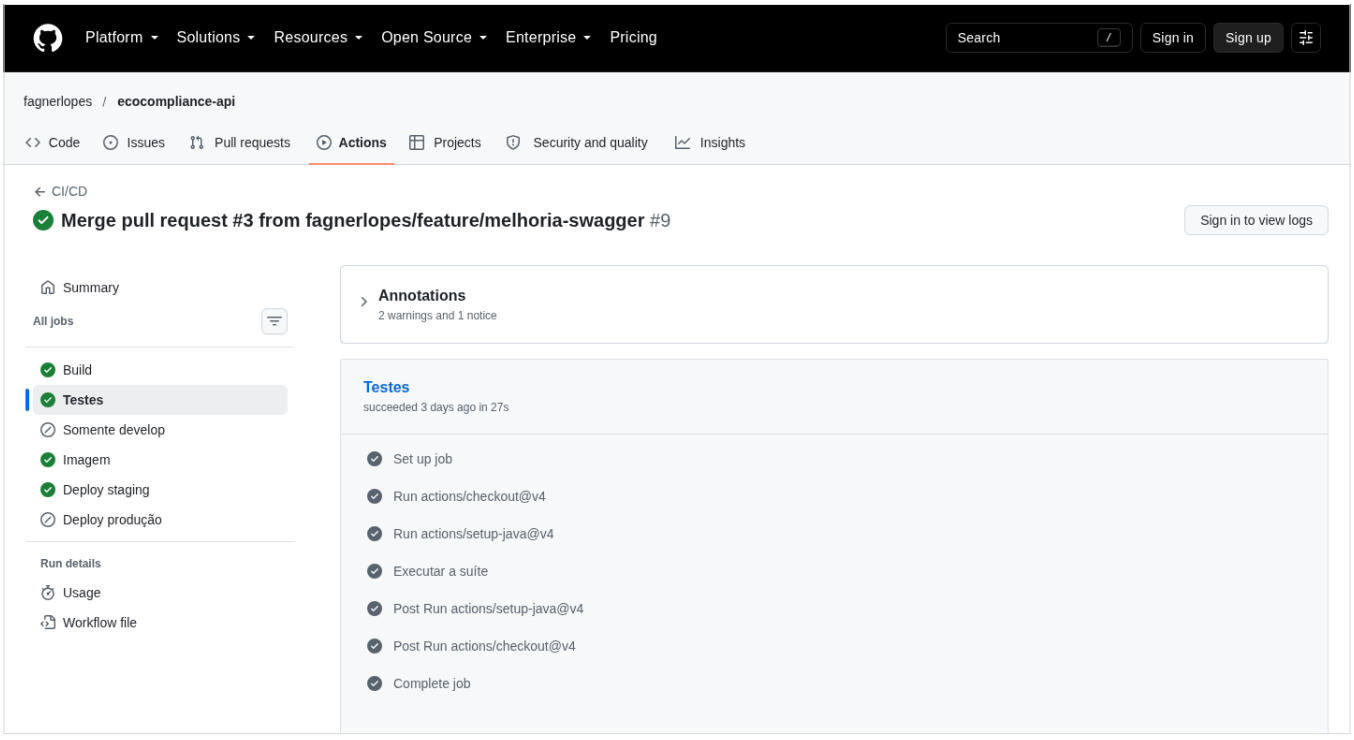


Figura 2. O mesmo push, no job Testes, passo Executar a suíte.

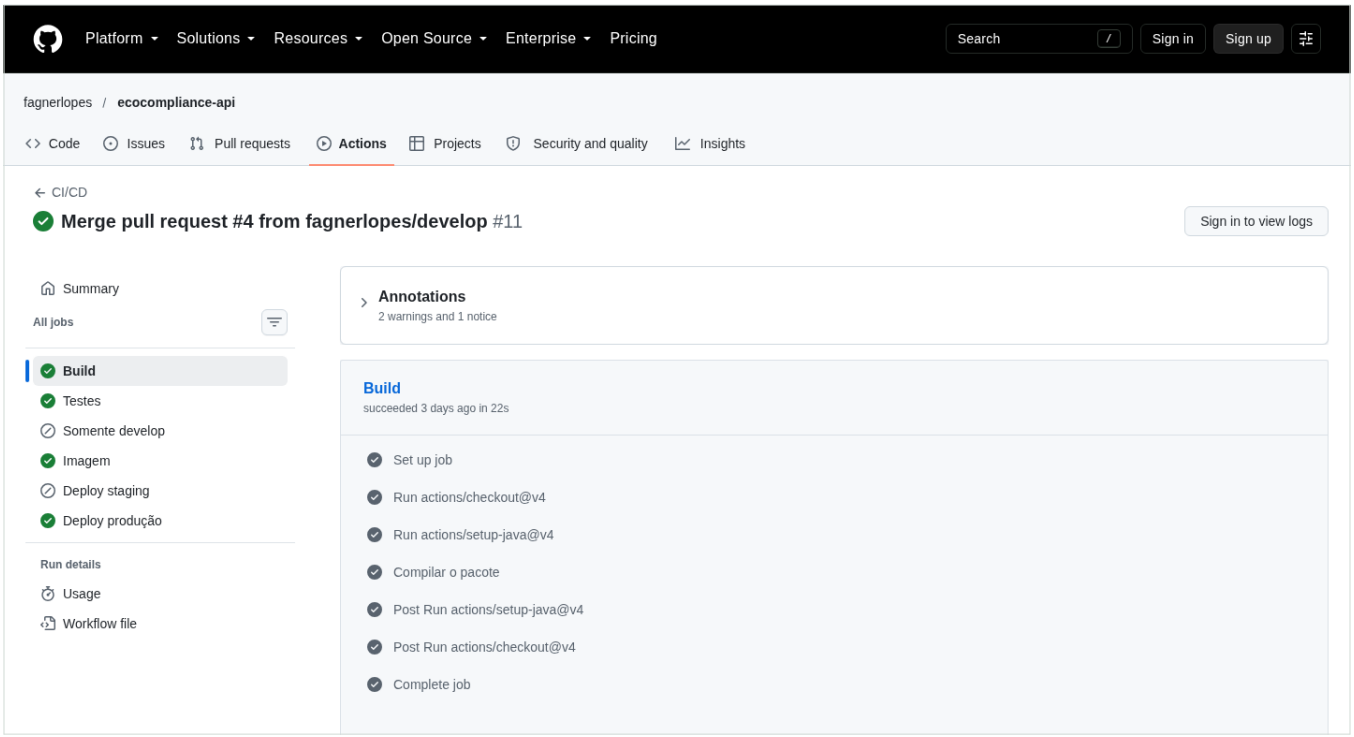


Figura 3. Push na main: Build, Testes, Imagem e Deploy produção concluídos.

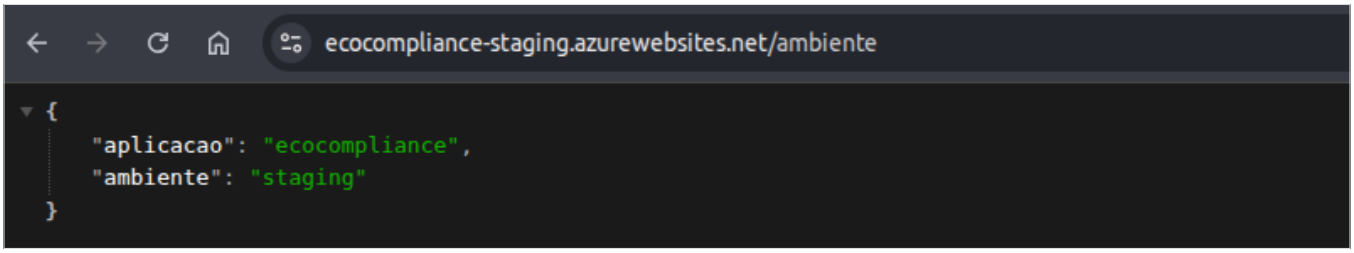


Figura 4. Staging em https://ecocompliance-staging.azurewebsites.net/ambiente.

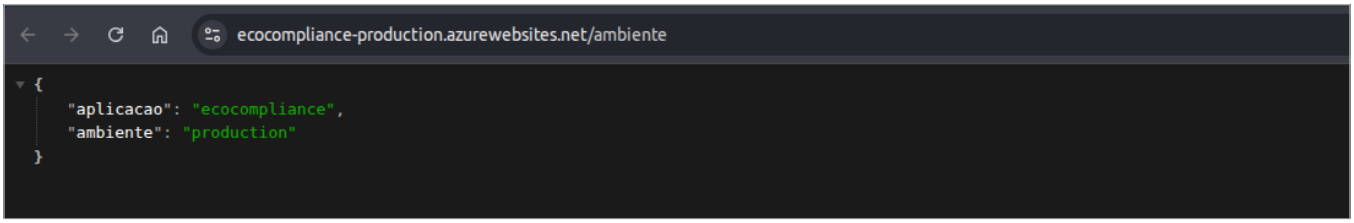


Figura 5. Produção em https://ecocompliance-production.azurewebsites.net/ambiente.

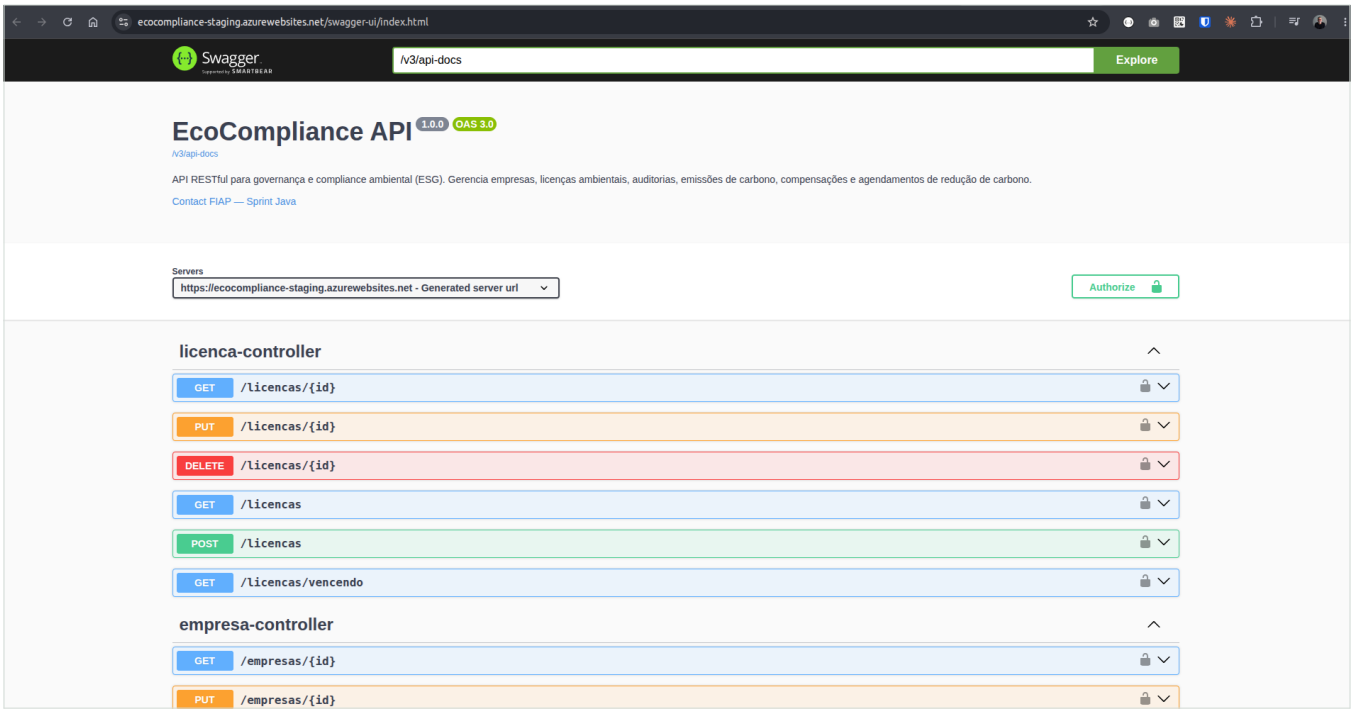


Figura 6. Swagger do staging em https://ecocompliance-staging.azurewebsites.net/swagger-ui/index.html.

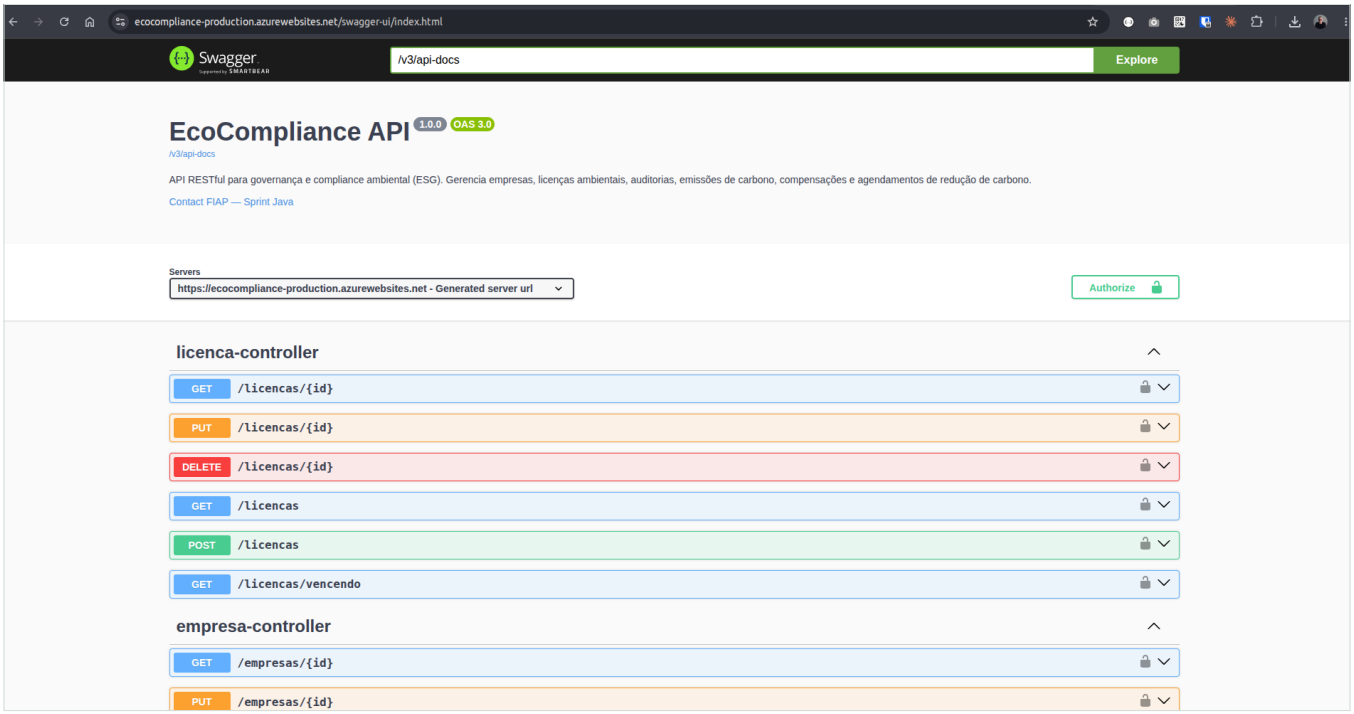


Figura 7. Swagger da produção em https://ecocompliance-production.azurewebsites.net/swagger-ui/index.html.

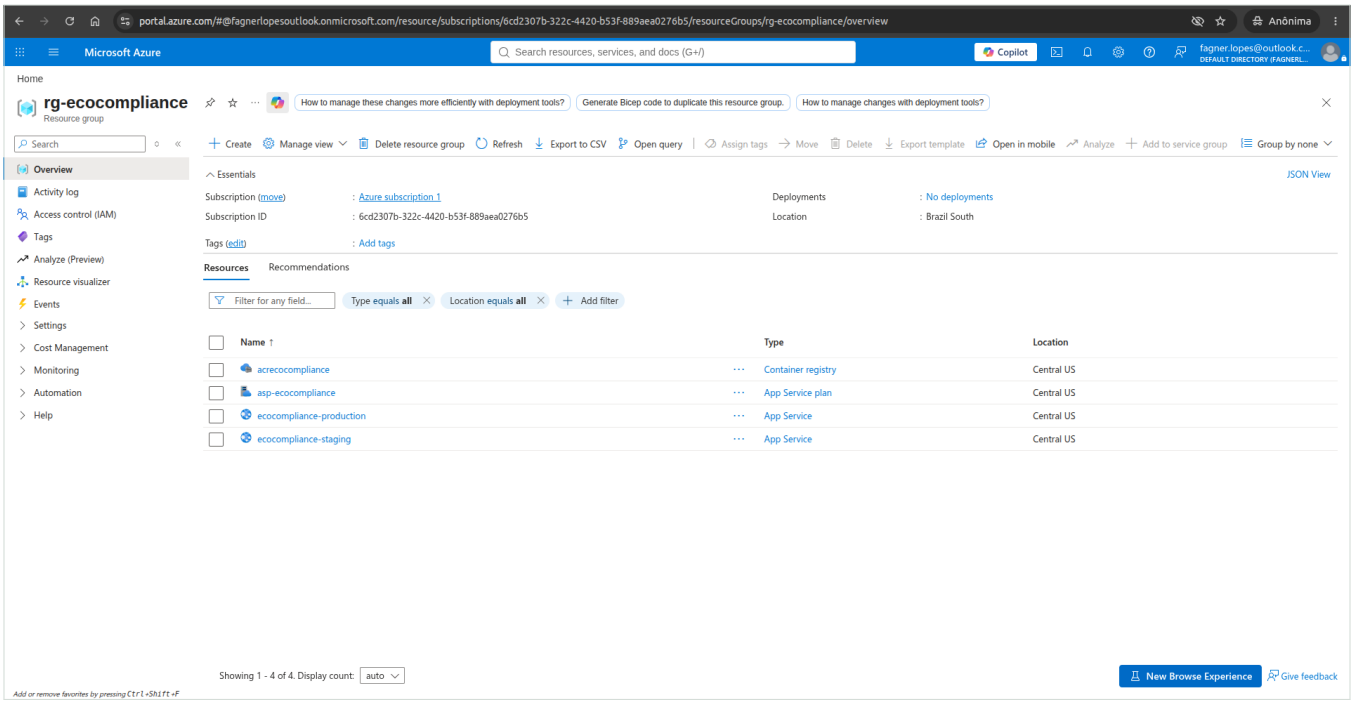


Figura 8. Grupo rg-ecocompliance: registry acrecocompliance, plano asp-ecocompliance e os dois App Services.

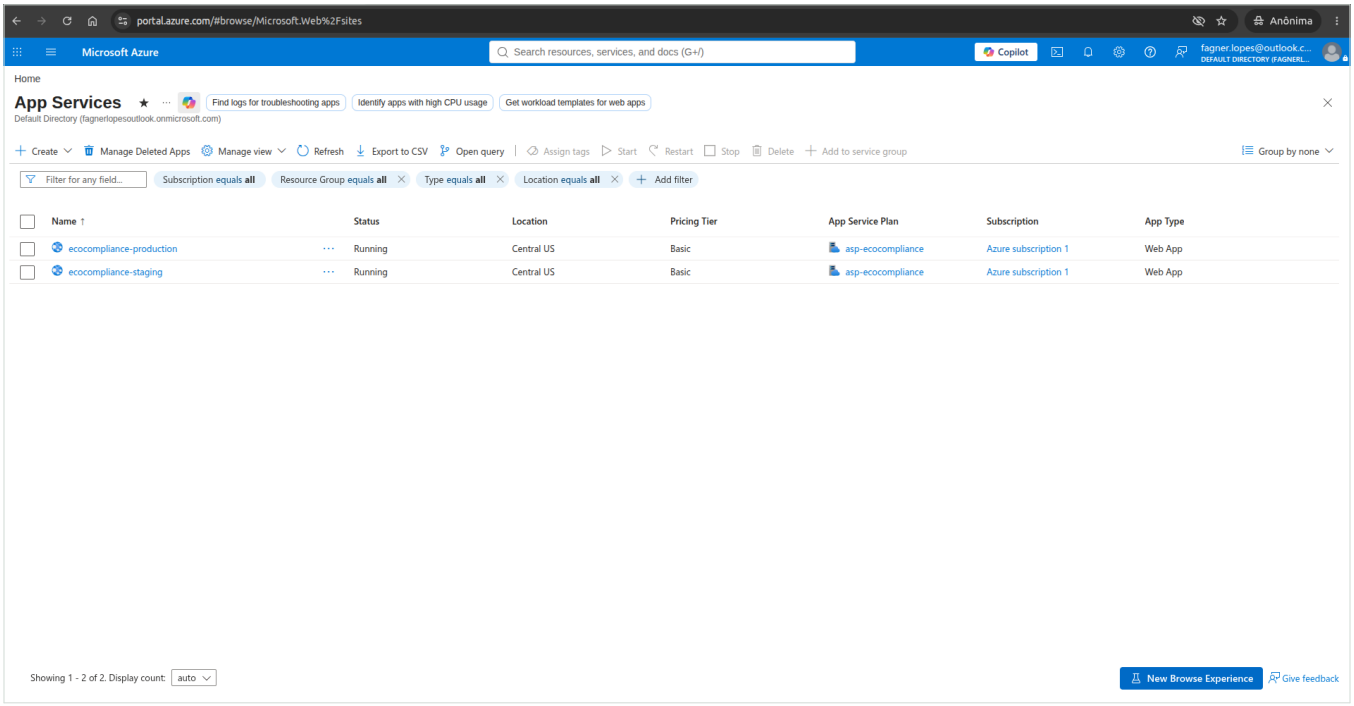


Figura 9. Os dois App Services no plano asp-ecocompliance, região Central US.

4. Desafios

• Dois Composes na mesma máquina se misturavam ao compartilhar volume e porta. Cada um passou a ter projeto, volume e rede próprios.

• A imagem publicada carrega o JRE e o jar. A suíte fica no job Testes, anterior à publicação da imagem.

• Um único workflow publica nos dois destinos. A branch decide o alvo: develop atualiza o staging e main atualiza a produção.

• O merge na main precisava depender da develop. O job Somente develop e a proteção da branch bloqueiam o merge quando o check falha, inclusive para o administrador do repositório.

5. Checklist

Item	OK
Projeto compactado em ZIP com estrutura organizada	sim
Dockerfile funcional, com explicação e exemplo	sim
docker-compose.yml, com explicação e exemplo	sim
Pipeline com build, teste e deploy	sim
README.md com instruções e evidências	sim
Documentação técnica (este PDF)	sim
Deploy em staging e produção	sim